

Google Cloudの仮想マシンで安全にはじめるOpenClaw ローカルマシンで実行するとなにが危ない？

OpenClawのようなAIエージェントは、自律的にコマンドを実行したりファイル进行操作したりする。これを**自分のPCで直接動かす**と、次のようなリスクがある。

- **ファイルの誤削除・上書き**：エージェントが意図せず大事なファイルを消してしまう
- **機密情報の漏洩**：ホームディレクトリや `.env` ファイルにあるパスワード・APIキーに触れてしまう
- **ウイルス・マルウェアの混入**：外部から取得したコードをそのまま実行する過程で悪意あるコードが動く可能性がある
- **リソースの圧迫**：AIの推論処理がCPU・メモリを食い尽くし、PCがフリーズする

「試しに動かしてみたら大事なプロジェクトのフォルダが消えた」という事故は珍しくない。安全に試すには、**本番環境から切り離された使い捨て可能な環境**を用意するのが鉄則だ。

ローカル、Docker、VM、VPSの違い

方式	概要	安全性	コスト	手軽さ
ローカル	自分のPCで直接実行	× 低い	無料	◎
Docker	コンテナで隔離して実行	△ まあまあ	無料～	○
VM（仮想マシン）	PC内に仮想PCを作って実行	○ 高い	無料～	△ 重い
VPS / クラウドVM	クラウド上の仮想サーバーで実行	◎ 最も高い	数十円～	○

Dockerは？ コンテナはある程度隔離されているが、設定によってはホストのファイルシステムをマウントしてしまい完全な隔離にならない。またコンテナ脱出の脆弱性が過去に見つかっていることもあり、未知のコードを動かすには不安が残る。

ローカルVMは？ VirtualBoxやUTMでVMを立てれば安全だが、PCのリソースを大量に消費するうえ、セットアップが複雑で初心者には敷居が高い。

結論: Google CloudのCompute Engineを使うのが安全

Terraformを利用することで、インフラ構成をテキストファイル（コード）として定義・自動構築できる。設定をテキストファイルに書き出すのは、まさに **「Infrastructure as Code (IaC)」** と呼ばれる非常に安全で優れたアプローチだ。事前に構成を確認（レビュー）でき

るため誤操作を完全に防ぎ、不要になった際もコマンド一つで全リソースを綺麗に一括削除できる。

今回はさらに安全性を高めるために、**完全に独立した専用ネットワーク (VPC)** を含む構成をTerraformで構築する。

環境構築

事前準備：GCPコンソール (Web UI) での作業

コマンドラインの前に、ブラウザでの作業3つとローカルPCへのTerraformインストールを済ませておく。

1. プロジェクトの作成

1. [Google Cloud Console](#) を開く
2. 画面上部の「プロジェクトを選択」→「新しいプロジェクト」
3. プロジェクト名を入力 (例: `openclaw-sandbox`) して「作成」
4. 作成後、**プロジェクトID** (例: `openclaw-sandbox-123456`) をメモしておく

プロジェクト名とプロジェクトIDは別物。IDはコンソール上部のプロジェクト選択欄で確認できる。

2. 課金設定の紐付け

作成したプロジェクトに請求先アカウントをリンクする。「お支払い」メニューから設定できる。新規アカウントなら \$300 分の無料クレジットが付与される。

3. APIの有効化

「APIとサービス」→「ライブラリ」から以下の2つを検索して有効化する。

- **Compute Engine API** : VMを作るために必要
- **Cloud Resource Manager API** : TerraformがGCPプロジェクトを操作するために必要

4. Terraformのインストール (ローカルPC)

Macの場合 (Homebrewを使う)

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

`brew tap` は新しいパッケージリポジトリを追加するコマンド。HashiCorp (Terraformの開発元) のリポジトリを追加してから `terraform` をインストールしている。

インストール確認

```
terraform version
```

バージョン番号が表示されればOK。

ステップ1：設定ファイル（main.tf）の作成

ローカルPCの任意の場所に作業用ディレクトリを作り(**openclaw-sandbox**といった名前のフォルダでOK)、その中に `main.tf` というテキストファイルを作成して以下のコードを貼り付ける。

注：下記にmain.tfをあらかじめ用意した。ダウンロードして作業用フォルダ内に配置してください。 https://drive.google.com/file/d/11mQiuI9Tgj4uxOvdThZ4tn2IEY_-YKzP/view?usp=sharing

```
mkdir openclaw-sandbox && cd openclaw-sandbox
```

`mkdir` はフォルダを作るコマンド。 `&&` は「前のコマンドが成功したら次も実行する」という意味。 `cd` はフォルダに移動するコマンド。

`main.tf` の内容：

※ `YOUR_PROJECT_ID` の部分だけ、Web UIで確認したプロジェクトIDに書き換えること。

```
provider "google" {
  project = "YOUR_PROJECT_ID" # ここを書き換える
  region  = "asia-northeast1"
  zone    = "asia-northeast1-a"
}

# 1. 完全に独立したカスタムVPCネットワーク
resource "google_compute_network" "vpc_network" {
  name                  = "openclaw-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "subnet" {
  name          = "openclaw-subnet"
  ip_cidr_range = "10.0.1.0/24"
  region       = "asia-northeast1"
  network      = google_compute_network.vpc_network.id
}
```

2. アウトバウンド用: Cloud Router & Cloud NAT

```
resource "google_compute_router" "router" {
  name      = "openclaw-router"
  network   = google_compute_network.vpc_network.id
  region    = "asia-northeast1"
}

resource "google_compute_router_nat" "nat" {
  name                               = "openclaw-nat"
  router                             = google_compute_router.router.name
  region                             =
google_compute_router.router.region
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
}
```

3. インバウンド用: ファイアウォール (IAP経由のSSHのみ許可)

```
resource "google_compute_firewall" "allow_iap_ssh" {
  name      = "allow-iap-ssh"
  network   = google_compute_network.vpc_network.id

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }

  source_ranges = ["35.235.240.0/20"] # IAPのIP帯域
}
```

4. VMインスタンス (外部IPなし)

```
resource "google_compute_instance" "vm_instance" {
  name          = "openclaw-vm"
  machine_type  = "e2-standard-2"
  zone          = "asia-northeast1-a"

  boot_disk {
    initialize_params {
      image = "ubuntu-os-cloud/ubuntu-2204-lts"
      size  = 30
      type  = "pd-balanced"
    }
  }
}

network_interface {
  network      = google_compute_network.vpc_network.id
  subnetwork   = google_compute_subnetwork.subnet.id
  # access_config ブロックを書かないことで外部IPを付与しない
}
}
```

この設定が何をしているか

カスタムVPC (`google_compute_network` / `google_compute_subnetwork`) GCPのデフォルトネットワークを使わず、OpenClaw専用の完全に独立したネットワーク空間を作る。他のシステムと完全に切り離された「専用の部屋」を用意するイメージ。

Cloud Router & Cloud NAT (`google_compute_router` / `google_compute_router_nat`) VMからインターネットへの**アウトバウンド通信のみ**を許可する仕組み。VMがパッケージをダウンロードしたり外部APIを叩いたりできるが、外部からVMに直接アクセスすることはできない。

ファイアウォール (`google_compute_firewall`) SSH (ポート22番) への接続を、`35.235.240.0/20` というIP帯域からのみ許可する。これはGoogleのIAP (Identity-Aware Proxy) のIPアドレス帯域であり、事実上「Googleの認証を通過した自分だけ」がSSH接続できるという意味になる。

VMインスタンス (`google_compute_instance`) `access_config` ブロックを**書かない**ことで、VMに外部IPアドレスが付与されない。外部から直接このVMに到達する手段がなく、攻撃の入り口を完全に塞いでいる。

ステップ2：構築の実行（ローカルのターミナル）

`main.tf` を保存したディレクトリで、以下のコマンドを順番に実行する。

GCPへの認証

```
gcloud auth application-default login
```

ブラウザが開いてGoogleアカウントへのログインを求められる。ログインすると、ローカルPC上にGCPを操作するための認証情報が保存される。

`application-default` とは「アプリケーション（ここではTerraform）がGCPにアクセスするときに使うデフォルト認証情報」という意味。Terraformはこの認証情報を使ってGCPを操作する。

Terraformの初期化

```
terraform init
```

`main.tf` を読み込み、必要なプラグイン（ここではGoogle Cloud用プロバイダー）をダウンロードする。初回は必ず実行すること。`.terraform/` というフォルダが作られ、そこにプラグインが格納される。

構築されるリソースの事前確認

```
terraform plan
```

実際には何も作らずに、これから何が作られるかをプレビューするコマンド。`+` が付いている行が新しく作成されるリソース。「こんなものが作られます、よいですか？」という確認の場面。実行前に必ずここで内容を確認する習慣をつけよう。

実際の構築

```
terraform apply
```

計画通りにGCP上にリソースを作成する。途中で `Enter a value:` と聞かれたら `yes` と入力してEnterキーを押す。

完了するとターミナルに `Apply complete!` と表示される。これで、外部から完全に隔離された安全なネットワークとVMが構築された。

ステップ3：VMへの接続

```
gcloud compute ssh openclaw-vm \
  --zone=asia-northeast1-a \
  --tunnel-through-iap \
  --project=YOUR_PROJECT_ID
```

各オプションの意味：

- `openclaw-vm`：接続先のVM名（`main.tf` で指定した `name`）
- `--zone=asia-northeast1-a`：VMが存在するゾーン（東京）
- `--tunnel-through-iap`：GoogleのIAP（Identity-Aware Proxy）経由でSSHトンネルを張る。これにより、VMに外部IPがなくてもSSH接続できる。インターネットに直接穴を開けずに済む、最も安全な接続方法。
- `--project=YOUR_PROJECT_ID`：使用するGCPプロジェクトのID

接続に成功するとターミナルのプロンプトが `username@openclaw-vm:~$` のようなものになる。ここからはVM内の操作になる。

接続に成功すると、ターミナルのプロンプトが `username@openclaw-vm:~$` になる。ここからはVM内の操作になる。ホストOS（手元のPC）への影響を一切気にせずに作業できる。

ステップ4：VM内でのセットアップ（OpenClawのインストール）

以下のコマンドを上から順にコピー＆ペーストして実行する。

システム更新と必須パッケージのインストール

まずUbuntuのパッケージリストを最新にし、Node.js（OpenClawの実行環境）やGitなどをインストールする。

```
# パッケージリストの更新と基本ツールのインストール
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential
```

- `sudo`：管理者権限でコマンドを実行する。Windowsの「管理者として実行」に相当。
- `apt update`：インストール可能なパッケージの一覧を最新化する。実際にはまだ何もインストールしない。
- `apt upgrade -y`：インストール済みのパッケージをまとめてアップデートする。`-y`は「確認なしでyesと答える」オプション。
- `apt install -y git curl build-essential`：3つのツールを一括インストール。`git`はソースコードの取得に、`curl`はURLからファイルをダウンロードするのに、`build-essential`はコンパイルに必要な基本ツール群。

```
# Node.js（バージョン24.x）のインストール
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs
```

Node.jsは `apt` の標準リポジトリに最新版が入っていないため、NodeSourceという配布元のセットアップスクリプトを先に実行してリポジトリを追加してからインストールする。OpenClawの動作要件はNode.js **22以上**、推奨は**24**。

- `curl -fsSL URL`：URLからファイルを取得する。`-f`は失敗時にエラーを出す、`-s`は進捗を非表示、`-S`はエラーは表示、`-L`はリダイレクトに追従するオプション。
- `| sudo -E bash -`：取得したスクリプトをそのまま管理者権限で実行する。`|`はパイプ（前のコマンドの出力を次のコマンドの入力に渡す）。

```
# インストール確認（バージョンが表示されればOK）
node -v
npm -v
```

`node -v` と `npm -v` でそれぞれのバージョン番号が表示されればインストール成功。
`v24.x.x` のように表示されるはず。

OpenClawの取得とセットアップ

npmからインストールする。

```
sudo npm install -g openclaw@latest
```

- `npm install -g` : 端末にグローバルに（どこでも使えるコマンドとして）インストールする

OpenClawのセットアップ

インストール後、以下の2ステップで初期設定を行う。

1. オンボーディングウィザードの実行

```
openclaw onboard --install-daemon
```

認証設定・Gateway設定・チャンネル構成をまとめて対話形式でセットアップするコマンド。`--install-daemon` オプションを付けることで、GatewayをOS起動時に自動起動するデーモンとして登録する。

2. Gatewayの起動確認

```
openclaw gateway status
```

Gatewayが正常に動いているかを確認する。`running` のような表示が出ればOK。

ステップ5：検証終了後のリソース一括削除

検証が終わって環境が不要になったら、ローカルPCのターミナルで以下のコマンドを実行するだけ。これを実行すると仮想マシンが消去される（OpenClawの設定もろとも全て消えます）。物理的にOpenClawを停止したい場合や、仮想マシンを破棄したい場合にのみ実行すること。

```
terraform destroy
```

`yes` と入力して確認すると、VM・NAT・ルーター・VPCなど、関連するすべての課金リソースが**確実かつ綺麗**に削除される。手作業でポチポチ削除する必要がなく、消し忘れによる予期せぬ請求を防ぐことができる。ここがTerraform（IaC）最大のメリット。

まとめ：コマンド一覧

ローカルPCで実行するコマンド

ステップ	コマンド	説明
認証	<code>gcloud auth application-default login</code>	GCPへログイン
初期化	<code>terraform init</code>	プラグインのダウンロード
確認	<code>terraform plan</code>	何が作られるかレビュー
構築	<code>terraform apply</code>	実際にVMを作成
接続	<code>gcloud compute ssh openclaw-vm --zone=asia-northeast1-a --tunnel-through-iap --project=YOUR_PROJECT_ID</code>	IAP経由でSSH
削除	<code>terraform destroy</code>	全リソースを一括削除・課金停止

VM内で実行するコマンド

ステップ	コマンド	説明
システム更新	<code>sudo apt update && sudo apt upgrade -y</code>	パッケージを最新化
基本ツール	<code>sudo apt install -y git curl build-essential</code>	Git・curl等をインストール
Node.js追加	<code>curl -fsSL https://deb.nodesource.com/setup_24.x \ sudo -E bash -</code>	Node.jsリポジトリを登録
Node.jsインストール	<code>sudo apt install -y nodejs</code>	Node.js本体をインストール
OpenClawインストール	<code>sudo npm install -g openclaw@latest</code>	OpenClawをグローバルインストール
初期設定	<code>openclaw onboard --install-daemon</code>	認証・Gateway設定をまとめて実行
起動確認	<code>openclaw gateway status</code>	Gatewayが動いているか確認

クラウドVM上でOpenClawを動かせば、ローカル環境に一切影響を与えずに安全に実験できる。何か壊れても `terraform destroy && terraform apply` で元通り。